

КОМП'ЮТЕРНА ОБРОБКА ЗОБРАЖЕНЬ

Digital Image Processing - DIP

Стиснення зображень

1. Загальні відомості зберігання зображень. Урахування особливостей зору.
2. Стиснення без витрат.
3. Стиснення з витратами.

Графічні формати

Графічний формат = спосіб організації інформації о зображенні в файлі

Інформація в графічному файлі

Не графічна

Розмір зображення

Версія формату

РАСТРОВІ
ЗОБРАЖЕННЯ

Графічна

Зберігання кольору
попільсьельно

Використання
палітри

Особливості зображень як типу даних

1) Зображення (як і відео) займають набагато більше місця в пам'яті, ніж текст, що визначає актуальність алгоритмів архівації графіки.

2) Людський зір при аналізі зображення оперує контурами, загальним переходом кольорів і порівняно нечутливо до малих змін в зображенні. Таким чином, можна створити ефективні алгоритми архівації зображень, в яких декомпресоване зображення не співпадатиме з оригіналом.

Особливості зображень як типу даних

3) Зображення на відміну, наприклад, від тексту володіє надмірністю в 2-х вимірах (як правило, сусідні точки, як по горизонталі, так і по вертикалі в зображенні близькі за кольором). Тому при створенні алгоритму компресії графіки завжди використовуються особливості структури зображення.

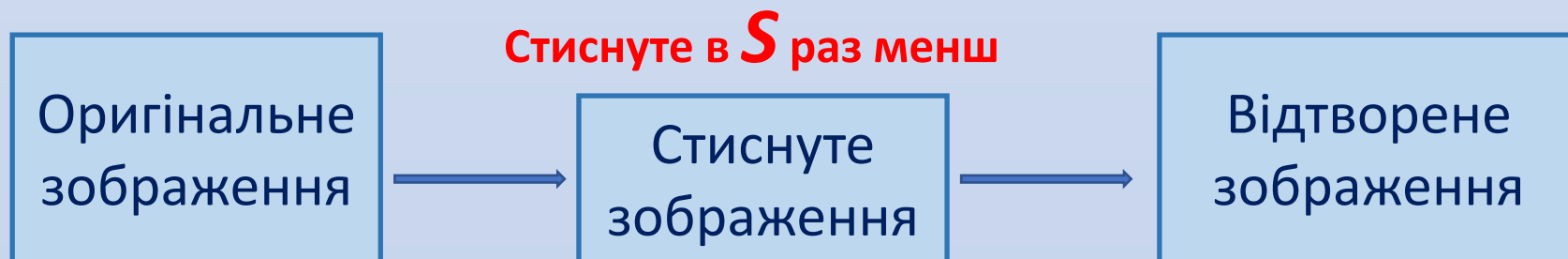
КОМПРЕСІЯ : СТИСНЕННЯ

Стиснення:

- Для зменшення обсягу інформації, що зберігається.
- Для зменшення втрат часу передачі інформації по каналах зв'язку.

Базова характеристика: коефіцієнт стиснення:

$$S = \frac{\text{обсяг інформації до стиснення}}{\text{обсяг інформації після стиснення}}$$



КОМПРЕСІЯ : СТИСНЕННЯ

Важливо: один і той же алгоритм може мати різні ефекти для різних зображень та при використанні різних додатків.

Класи додатків, що використовують алгоритми компресії

1. Додатки з високими вимогами до часу архівації та розархівації, які передбачають перегляд зменшеної копії зображення і пошук в базі даних зображень.

- Приклади: Видавничі системи та інформаційні вузли в WWW. Працюють з повноколірними зображеннями різного розміру і з великими двоколірними зображеннями.

Класи додатків, що використовують алгоритми компресії

2. Додатки з високими вимогами до ступеня архівації та часу розархівації (час архівації ролі не грає, застосовувані алгоритми істотно асиметричні за часом).

- Приклад: Довідники та енциклопедії на CD.

Класи додатків, що використовують алгоритми компресії

3) Додатки з дуже високими вимогами до ступеня архівації.

Приклад: Internet (WWW).

Оскільки при цьому навантаження на процесор низьке, то тут можуть знайти застосування ефективно стискаючі складні алгоритми з порівняно великим часом розархівації.

Вимоги до алгоритмів стиснення

- Високий ступінь компресії.
- Висока якість декомпресованого зображення.
- Висока швидкість компресії.
- Висока швидкість декомпресії.
- Масштабування зображень (легкість зміни розмірів зображення до розмірів вікна активного додатку).
- Можливість показати огрублене зображення (низької роздільної здатності), використавши тільки початок файлу.

Вимоги до алгоритмів стиснення

- Стійкість до помилок (локальність порушень в зображенні при псуванні або втраті фрагмента файлу, що передається).
- Невелика вартість апаратної реалізації.
Ефективність програмної реалізації.

Критерії порівняння алгоритмів

1. Коефіцієнти стиснення: гірший, середній і кращий.

- **Гірший** - частка, на яку зросте зображення, якщо вихідні дані будуть найгіршими.
- **Середній** - середньостатистичний коефіцієнт для того класу зображень, на який орієнтований алгоритм.
- **Кращий** - показує ступінь стиснення найкращого (як правило, абсолютно чорного) зображення, іноді фіксованого розміру.

Критерії порівняння алгоритмів

2. Клас зображень, на який орієнтований алгоритм.

3. Симетричність. Відношення характеристики алгоритму кодування до аналогічної характеристики при декодуванні. Характеризує ресурсомісткість процесів кодування і декодування. Найбільш важливі симетричність за часом (відношення часу кодування до часу декодування) і по пам'яті.

4. Втрати якості.

5. Характерні особливості алгоритму і зображень, до яких його застосовують.

Класифікація алгоритмів стиснення

Алгоритми стиснення

Алгоритми без втрати
інформації

lossless - встановлене
зображення повністю
співпадає з
оригіналом

Алгоритми з втратою
інформації

lossy - встановлене
зображення
відрізняється від
оригіналу

Стиснення зображень

- 1. Загальні відомості зберігання зображень. Урахування особливостей зору.**
- 2. Стиснення без витрат.**
- 3. Стиснення з витратами.**

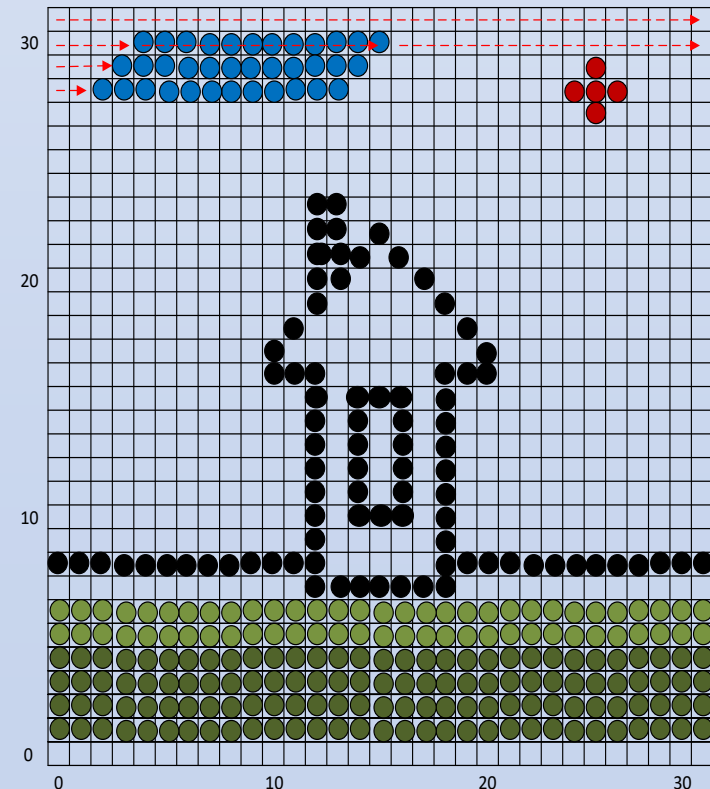
Алгоритм групового кодування (RLE - Run Length Encoding)

Ланцюжок однакових байтів



Лічильник повторень : RGB

Перший варіант
алгоритму - РСХ



Ознака лічильника - одиниці в двох верхніх бітах зчитаного байта:



Другий варіант алгоритму (TIFF, TGA)

Лічильник повторень : RGB

Лічильник (8 біт)

0 XXXXXXXX

Пропуск

Лічильник (8 біт)

1 XXXXXXXX

RGB

Характеристики алгоритму RLE:

Коефіцієнти компресії:

	Кращий	середній	гірший
1 варіант:	32,	2,	0,5.
2 варіант:	64,	3,	128/129.

Клас зображень: Зображення з невеликою кількістю кольорів (ділова і наукова графіка).

Симетричність: Близько 1.

Алгоритм кодування Хаффмана

Класичний алгоритм, відомий з 60-х років ХХ століття.

Зіставляє символам вхідного потоку, які зустрічаються більшу кількість разів, ланцюжок біт меншої довжини, і навпаки.

Для збору статистики вимагає двох проходів по зображенню.

Алгоритм кодування Хаффмана

Визначення. Нехай заданий алфавіт A

$$A = \{\alpha_1, \alpha_2, \dots, \alpha_n\}$$

що складається з кінцевого числа n різних символів. Алфавіту відповідає множина ваг W

$$W = \{w_1, w_2, \dots, w_n\}$$

цілих позитивних чисел – визначених як кількість w_i символів α_i в деякому тексті (зображенні).

Код Хаффмана $C = \{c_1, c_2, \dots, c_n\}$, де c_i – код символу α_i , є такий що:

- c_i – не є префіксом для c_j при $c_i \neq c_j$,
- сума $\sum_{i \in [1, n]} w_i * |c_i|$ мінімальна, де $|c_i|$ – довжина L коду c_i .

Алгоритм побудови кодуєчої схеми

Крок 0. Розрахунок ваг (ймовірностей) w_i появи букв вхідного алфавіту α_i у вхідному потоці.

Крок 1. Впорядкування всіх букв вхідного алфавіту в порядку убутання ваг (ймовірностей) w_i .

Алгоритм побудови кодуєчої схеми

Крок 2. Об'єднуються два символи $\alpha_{(r-1)}$ і $\alpha_{(r)}$ з найменшими вагами $w_{(r-1)}$ і $w_{(r)}$ в один псевдосимвол $\alpha^r\{\alpha_{(r-1)}, \alpha_{(r-1)}\}$ з вагою $(w_{(r-1)} + w_{(r)})$
!! Допикується:

0 в початок коду $\mathbf{c}_{(r-1)} = "0" + \mathbf{c}_{(r-1)}$

1 в початок коду $\mathbf{c}_{(r)} = "1" + \mathbf{c}_{(r)}$

Алгоритм побудови кодуєчої схеми

Крок 3. Видалення зі списку
впорядкованих символів $\alpha_{(r-1)}$ і $\alpha_{(r)}$,
та занесення туди псевдосимволу
 $\alpha^r\{\alpha_{(r-1)}, \alpha_{(r-1)}\}$

Кроки 2, 3 повторюються до тих пір,
поки в списку не залишиться один
псевдосимвол (з вагою = 1).

Алгоритм побудови кодуєчої схеми (приклад)

Нехай є 6 букв в алфавіті:

з вагами :

$$a_1 \rightarrow w_1 = 0,4 ; a_2 \rightarrow w_2 = 0,25;$$

$$a_3 \rightarrow w_3 = 0,15 ; a_4 \rightarrow w_4 = 0,1;$$

$$a_5 \rightarrow w_5 = 0,06 ; a_6 \rightarrow w_6 = 0,04;$$

Тоді процес побудови схеми можна уявити
так:

Алгоритм побудови кодуєчої схеми (приклад)

Сортуємо символи на кожному кроці

Символи	Ваги символів					
a_1	0,40	0,40	0,40	0,40	0,60	1,00
a_2	0,25	0,25	0,25	0,35	0,40	
a_3	0,15	0,15	0,20	0,25		
a_4	0,10	0,10	0,15			
a_5	0,06	0,10				
a_6	0,04					

Будуємо кодове бінарне дерево

Алгоритм побудови кодуєчої схеми (приклад)

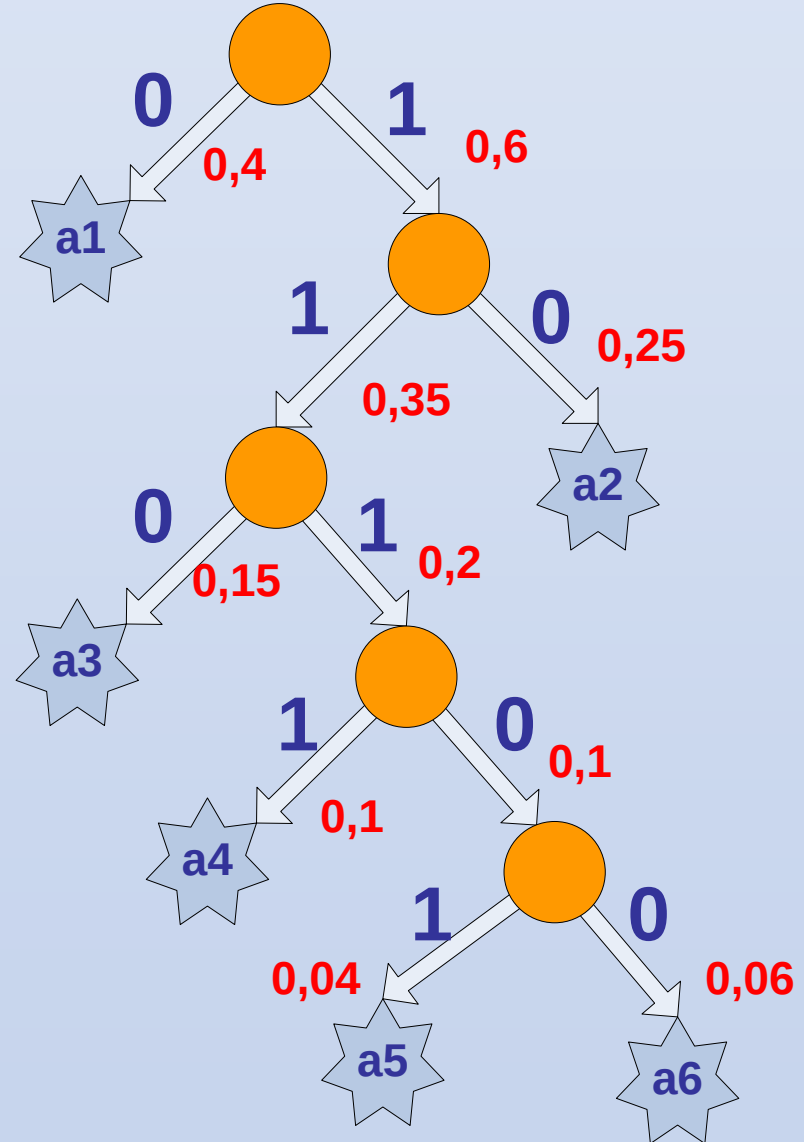
Для побудови кодуєчих слів треба пройти по стрілках від початкових символів до кінця отриманого бінарного дерева:

a1 = 0, a2 = 10

$$a_3 = 110, a_4 = 1111$$

a5 – 11101

a6 – 11100



Алгоритм кодування Хаффмана

При стисненні зображень в якості букв вхідного алфавіту використовуються кольори нестиснутого зображення

Приклад. Зображення 100x100, 8біт / піксель

Загальний обсяг 80000 біт колірної інформації.

У зображенні:

5000 синіх точок ($w_1=0.5$)

2400 червоних точок ($w_2=0.24$)

1500 жовтих точок ($w_3=0.15$)

1100 зелених точок ($w_4=0.11$)

Алгоритм кодування Хаффмана

Кодування:

Символи	Ваги символів				
col_1	0,5	0,50	0,5 0	Синій	0
col_2	0,24	0,26 0	0,5 1	Червоний	11
col_3	0,15 0	0,24 1		Жовтий	100
col_4	0,11 1			Зелений	101

Обсяг колірної інформації (біт):

$$5000 + 2400 * 2 + (1500 + 1100) * 3 = 17\,600 \text{ біт}$$

замість 80 000

Характеристики класичного алгоритму кодування Хаффмана:

Коефіцієнти компресії:

кращий - 8,

середній - 1,5,

гірший - 1

Симетричність: 2

Алгоритм Хаффмана

Алгоритм Хаффмана - єдиний з усіх не збільшує розміру вихідних даних в гіршому випадку (якщо не брати до уваги необхідність зберігати таблицю перекодування разом з файлом).

Практично не застосовується до зображень в чистому вигляді.

Зазвичай використовується як один з етапів компресії в більш складних схемах.

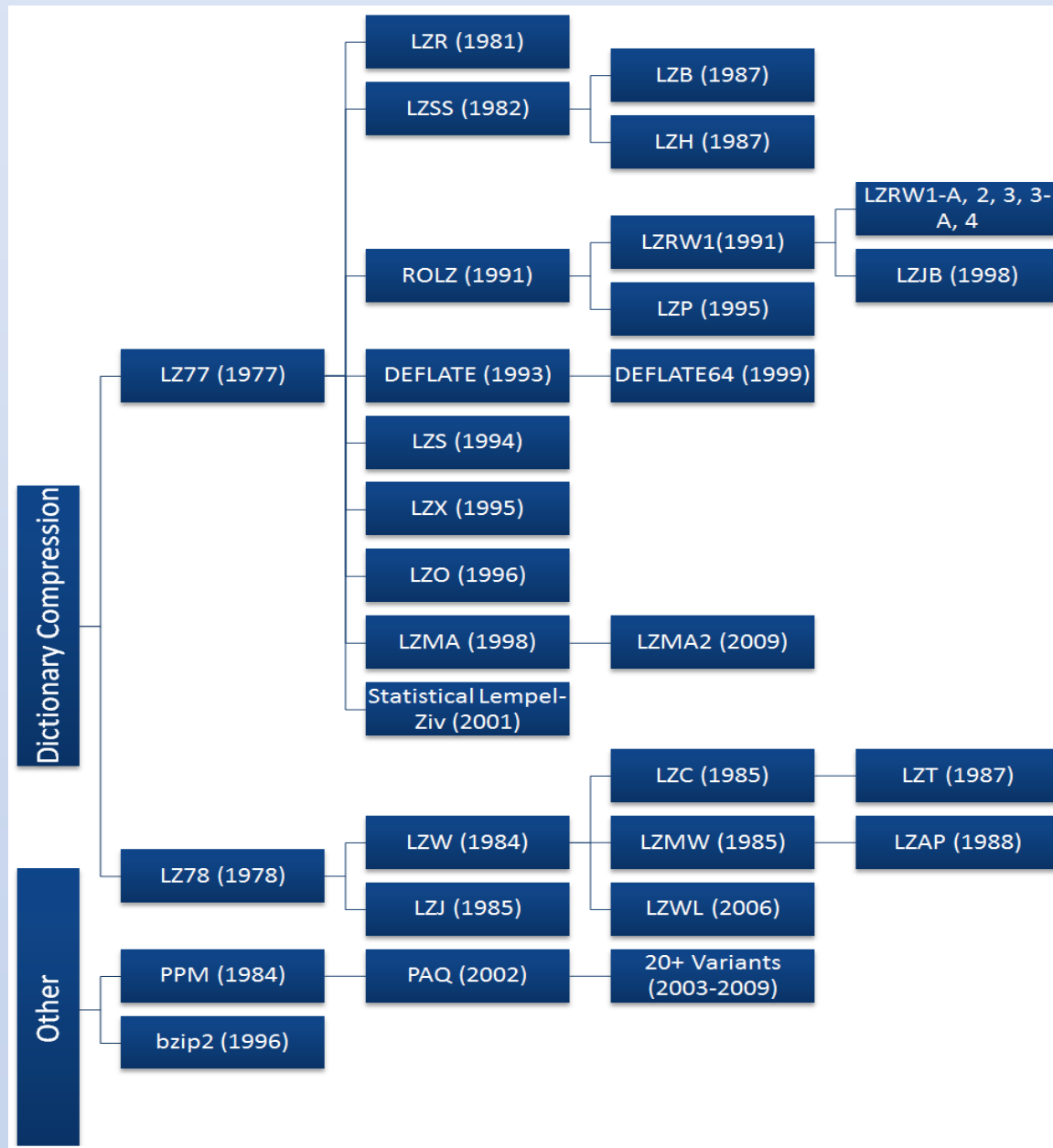
Алгоритм LZW (розробники Lempel, Ziv і Welch).

Стиснення здійснюється за рахунок однакових ланцюжків байт.

LZW універсальний - саме його варіанти використовуються в звичайних архіваторах.

LZW реалізований в форматах GIF і TIFF.

Родина LZW



Характеристики алгоритму LZW:

Коефіцієнти компресії:

Кращий	середній	гірший
--------	----------	--------

1000	57	4
------	----	---

Симетричність: Майже симетричний, за умови оптимальної реалізації

Алгоритм CCITT Group 3

При стисненні чорно-білих зображень (один біт на піксель) використовується модифікація алгоритму, яка називається CCITT Group 3 (Consultative Committee International Telegraph and Telephone - третя група по стандартизації Міжнародного Консультаційного Комітету з телеграфного й телефонного зв'язку).

Коефіцієнти компресії: в кращому випадку - до 213 , в середньому - 2, в гіршому випадку збільшує файл в 5 разів.

Симетричність: Близька до 1.

Клас зображень: чорно-білі зображення, із значно переважним білим (тексти).

Основні формати стиснених без втрат зображень

PNG (Portable Network Graphics) - це формат зображення без втрат, який може бути використаний як архіватор для стиснення зображень. PNG підтримує прозорість, що робить його хорошим вибором для веб-зображень.

GIF (Graphics Interchange Format) - без втрат і може використовуватись як архіватор для стиснення зображень. Однак він не підтримує стільки кольорів, як PNG, і не підтримує прозорість так само.

Основні формати стиснених без втрат зображень

TIFF (Tagged Image File Format) – це формат зображення без втрат, який широко використовується у професійній фотографії та друку. Він підтримує високу якість зображення та безліч кольорів.

BMF (Bitmap) - це формат зображення без втрат, який використовується в операційних системах Windows. Він підтримує високу якість зображення, але може займати велику кількість місця.

Основні формати стиснених без втрат зображень

WebP - відносно новий формат зображення, створений Google. Він забезпечує високу якість зображення та гарний ступінь стиснення, що робить його хорошим вибором для веб-зображень.

The END 11